

Convolutional Neural Networks for NLP

Text and Web Mining (IS6751)

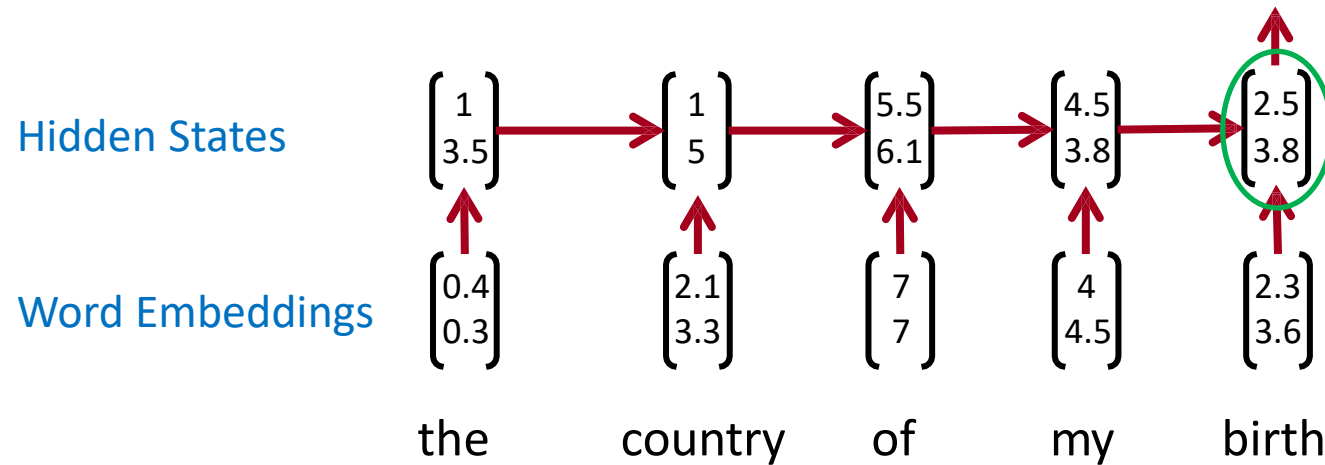
School of Communication and Information

Topics we will cover:

- Introduction to a convolution concept
- Introduction to Convolutional Neural Networks
- Convolutional Neural Networks for text classification

From RNNs to Convolutional Neural Nets

- Recurrent neural nets often capture too much of last words in final vector.



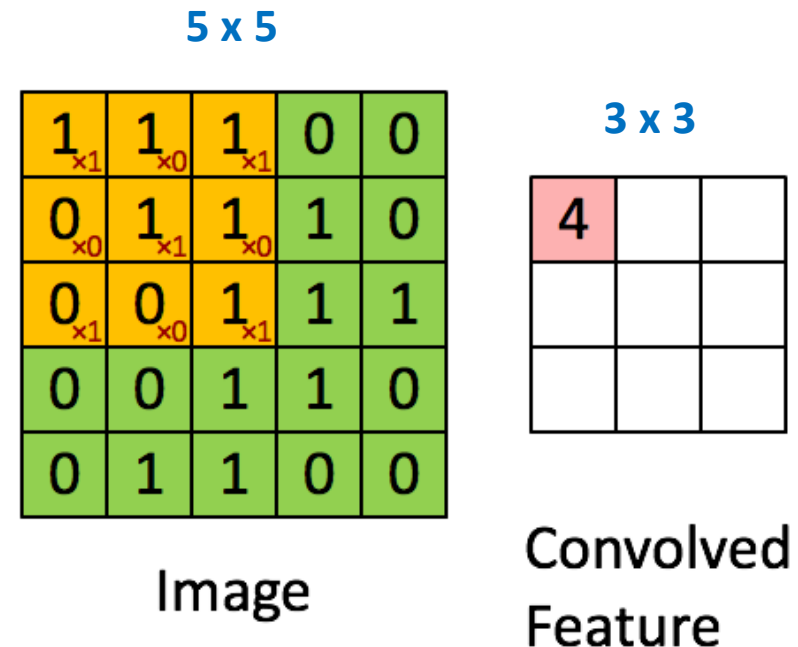
- Softmax is often only calculated at the last step for classification.

From RNNs to Convolutional Neural Nets

- Main CNN/ConvNet idea for NLP:
- What if we compute **vectors** for every possible word subsequence of a certain length, i.e., **n-grams**?
- Example: “tentative deal reached to keep government open” computes **vectors** for: **tri-grams**
 - "tentative deal reached", "deal reached to", "reached to keep", "to keep government", "keep government open"
- Regardless of whether phrase is grammatical.
- Not very linguistically or cognitively plausible.
- Then group them afterwards (more soon).

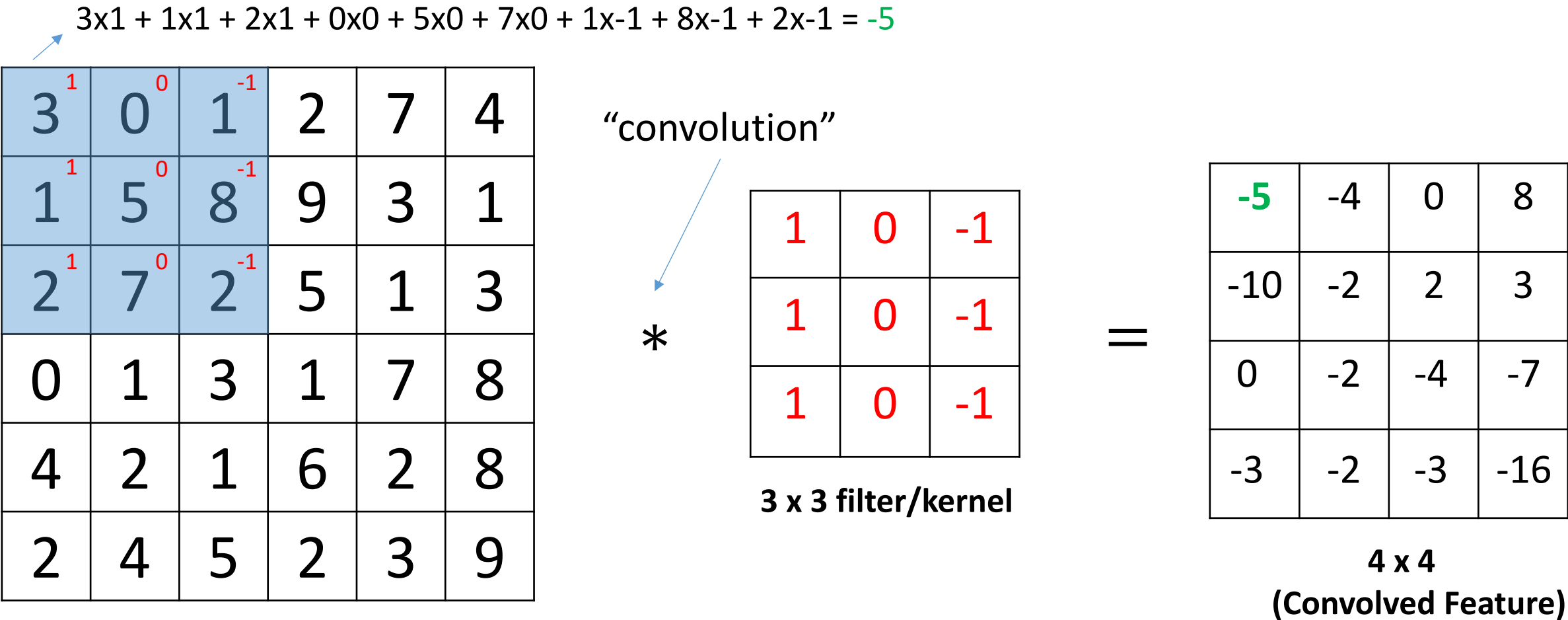
What is a convolution anyway?

- Convolution is classically used to extract features from images.
 - Models position-invariant identification (e.g., balls in various positions).
- 2d example ->
 - Green shows input.
 - Yellow color and red numbers show filter (=kernel) weights.
 - Pink shows output.



From Stanford UFLDL wiki

Convolution



6 x 6 (Grey image: 6 x 6 x 1)

4 x 4
(Convolved Feature)

Convolution

$0 \times 1 + 5 \times 1 + 7 \times 1 + 1 \times 0 + 8 \times 0 + 2 \times 0 + 2 \times -1 + 9 \times -1 + 5 \times -1 = -4$

3	0 ¹	1 ⁰	2 ⁻¹	7	4
1	5 ¹	8 ⁰	9 ⁻¹	3	1
2	7 ¹	2 ⁰	5 ⁻¹	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6 x 6 (Gray image: 6 x 6 x 1)

“convolution”

*

1	0	-1
1	0	-1
1	0	-1

3 x 3 filter/kernel

=

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

4 x 4

(Convolved Feature)

Introduction to Convolutional Neural Networks

Deep Learning on large images: Image Classification



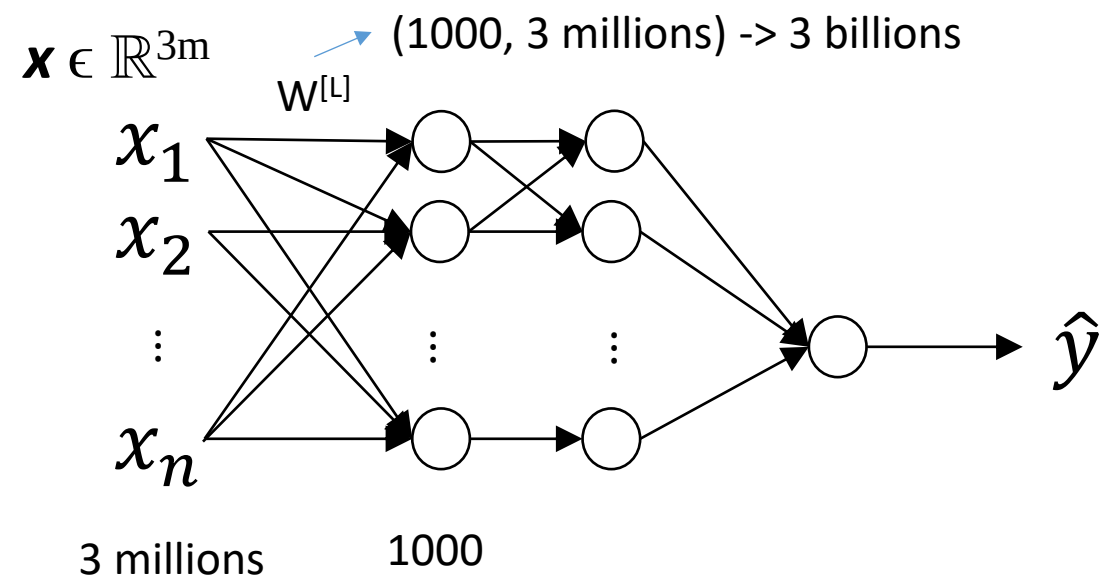
→ Cat? (0/1)

64x64x3 = 12,288 features



1000x1000x3 = 3 millions

Note: Fully connected neural networks can handle only small images.



Computer Vision Problem

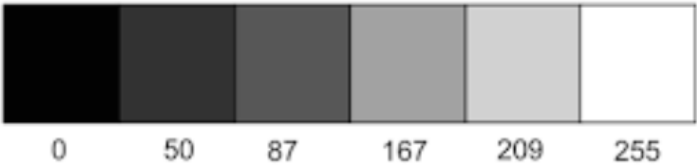


vertical edges



horizontal edges

Vertical edge detection



Note: higher numbers means lighter colours.

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

6 x 6

*

1	0	-1
1	0	-1
1	0	-1

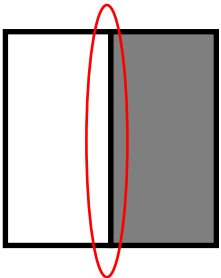
3 x 3

Vertical Filter

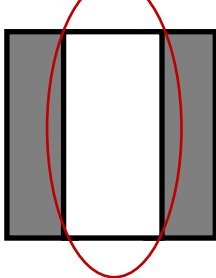
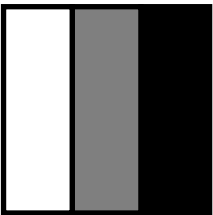
=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

4 x 4



*



Detected edge

Vertical and Horizontal Edge Detection

1	0	-1
1	0	-1
1	0	-1

Vertical Filter

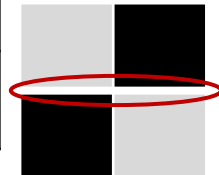
1	1	1
0	0	0
-1	-1	-1

Horizontal Filter

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10

6x6

*



1	1	1
0	0	0
-1	-1	-1

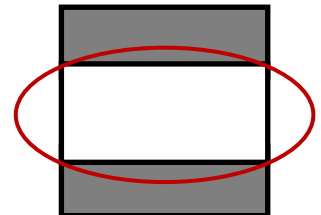
3 x 3

=

0	0	0	0
30	10	-10	-30
30	10	-10	-30
0	0	0	0

4 x 4

Detected edge



Learning to detect edges

1	0	-1
1	0	-1
1	0	-1

Vertical

Other
vertical
filters

1	0	-1
2	0	-2
1	0	-1

Sobel Filter

3	0	-3
10	0	-10
3	0	-3

Scharr Filter

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6x6

Convolution

*

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

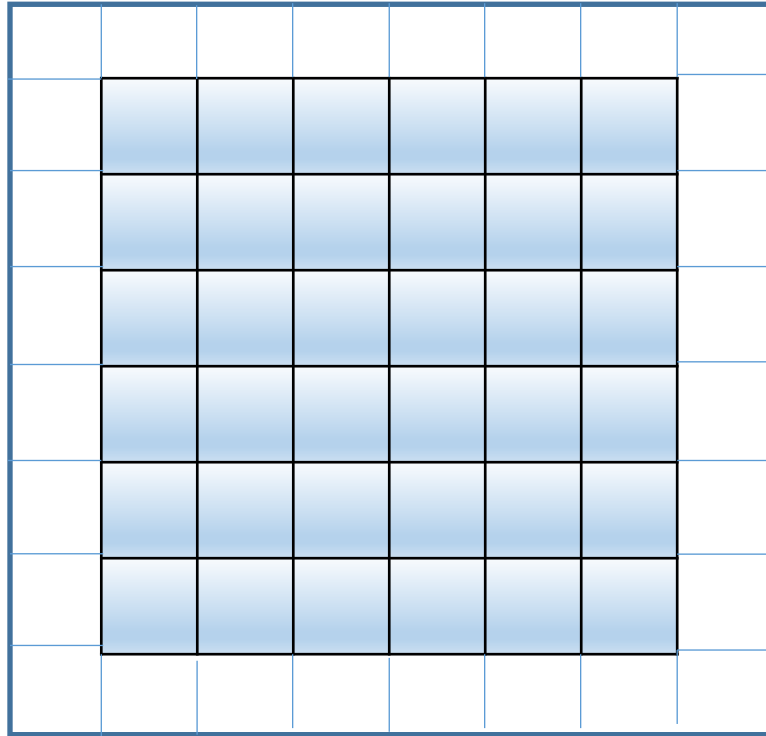
3x3

=

4 x 4

Learning 9 parameters instead of using fixed values: use backpropagation to learn weights.

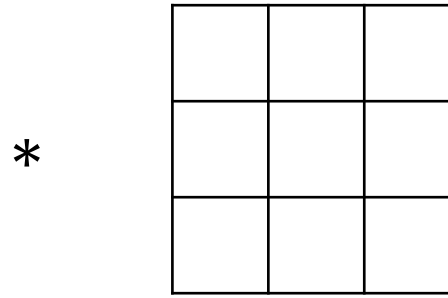
Padding



6 x 6 -> **8 x 8** (with padding=1)
 $n \times n$

Without Padding:

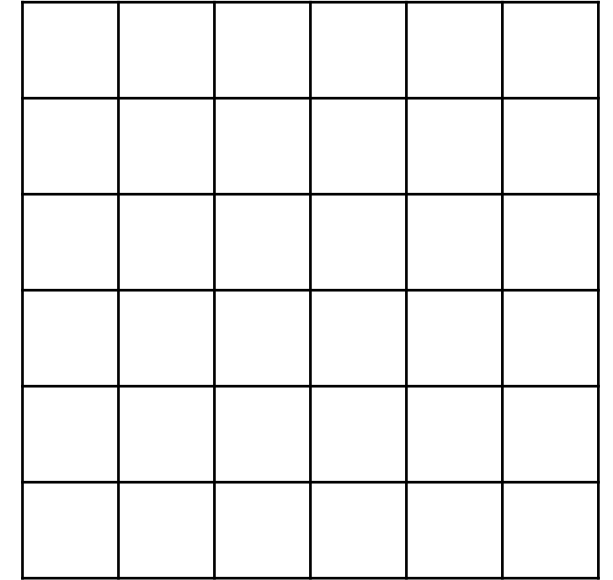
- Shrinking output
- Throw away information from edge



3 x 3
 $f \times f$

*

=



6 x 6

Output without padding: $(n - f + 1) \times (n - f + 1)$
 $(6 - 3 + 1) \times (6 - 3 + 1) = \mathbf{4 \times 4}$

Output with padding=1: $(n + 2p - f + 1) \times (n + 2p - f + 1)$
 $(6 + 2 - 3 + 1) \times (6 + 2 - 3 + 1) = \mathbf{6 \times 6}$

Valid and Same convolutions

No Padding

“Valid”: $n \times n * f \times f \rightarrow (n - f + 1) \times (n - f + 1)$
 $6 \times 6 * 3 \times 3 \rightarrow 4 \times 4$

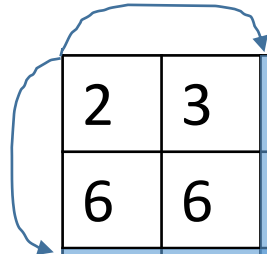
“Same”: Pad so that output size is the same as the input size.

$$(n + 2p - f + 1) \times (n + 2p - f + 1)$$

Since wanting $n + 2p - f + 1 == n$, $p = (f - 1) / 2$

So, when a filter is 3x3, $p = 1$; when a filter is 5x5, $p = 2$

Strided convolution



2	3	7 ³	4 ⁴	6 ⁴	2	9
6	6	9 ¹	8 ⁰	7 ²	4	3
3 ³	4 ⁴	8 ⁴	3 ⁰	8 ³	9	7
7 ¹	8 ⁰	3 ²	6	6	3	4
4 ⁻¹	2 ⁰	1 ³	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7x7

when
n x n * f x f

Padding p=0, **stride s=2**

Stride is the number of pixels shifts over the input matrix. When the **stride** is 1 then we move the filters to 1 pixel at a time. When the **stride** is 2 then we move the filters to 2 pixels at a time and so on.

步幅是输入矩阵上移动的像素数。当步长为1时，我们将过滤器一次移动到1个像素。当步长为2时，我们一次将滤镜移动到2个像素，以此以此地。

3	4	4
1	0	2
-1	0	3

3x3

=

91	100	83
69	91	127
44	72	74

3x3

Floor function (
e.g., 2.9 becomes 2.)

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

$$= (7+0-3)/2 + 1 = 3$$

Summary of convolutions

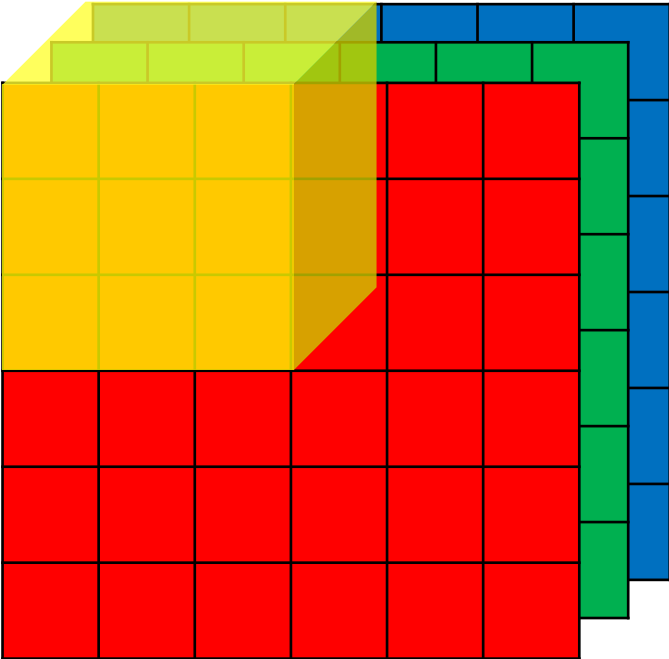
$n \times n$ image $f \times f$ filter

padding p stride s

Output Size:

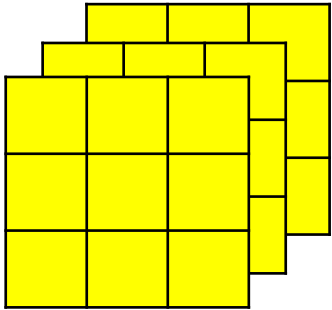
$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

Convolutions on RGB image



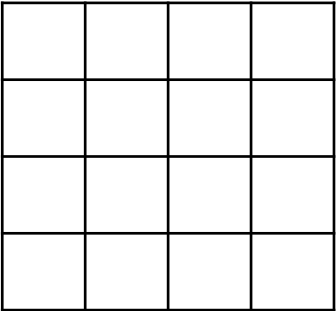
6x6x3

*



3x3x3

=



4 x 4

Vertical Edge
Filter for R

R

1	0	-1
1	0	-1
1	0	-1

G

0	0	0
0	0	0
0	0	0

B

0	0	0
0	0	0
0	0	0

3x3x3

Vertical Edge
Filters for R, G,
and B

R

1	0	-1
1	0	-1
1	0	-1

G

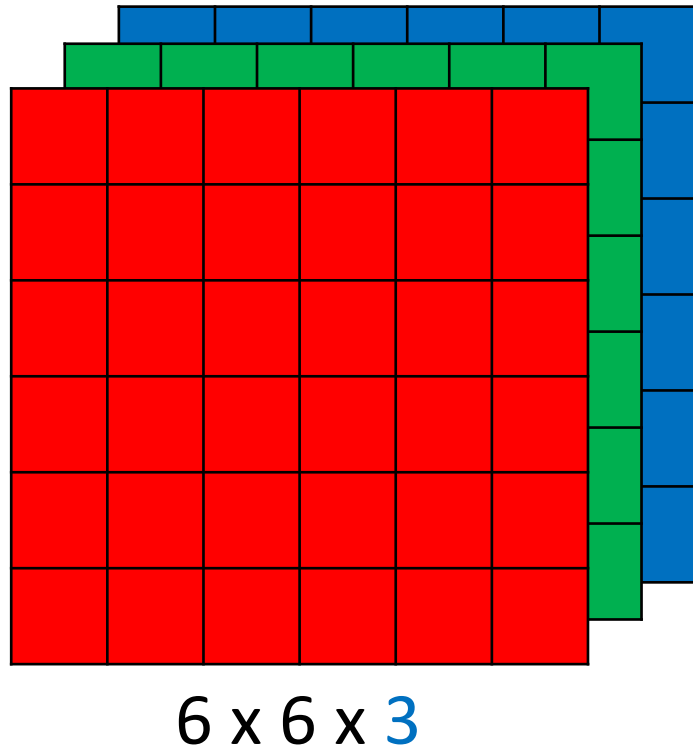
1	0	-1
1	0	-1
1	0	-1

B

1	0	-1
1	0	-1
1	0	-1

3x3x3

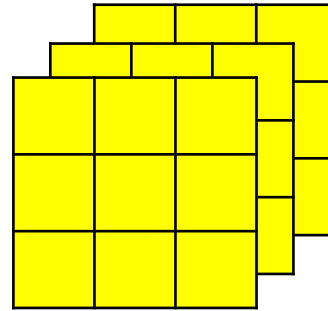
Multiple filters



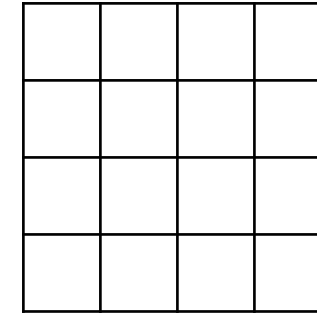
2 Filters

Vertical Edge

*

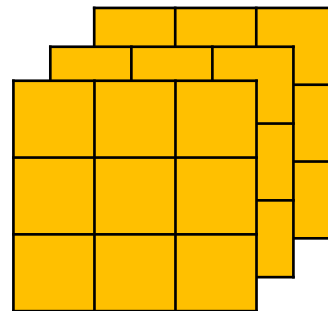


=

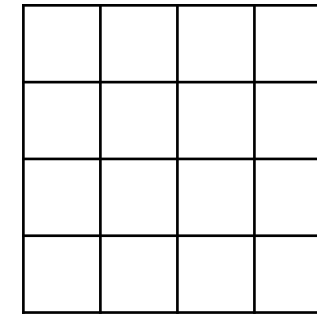


Horizontal Edge

*

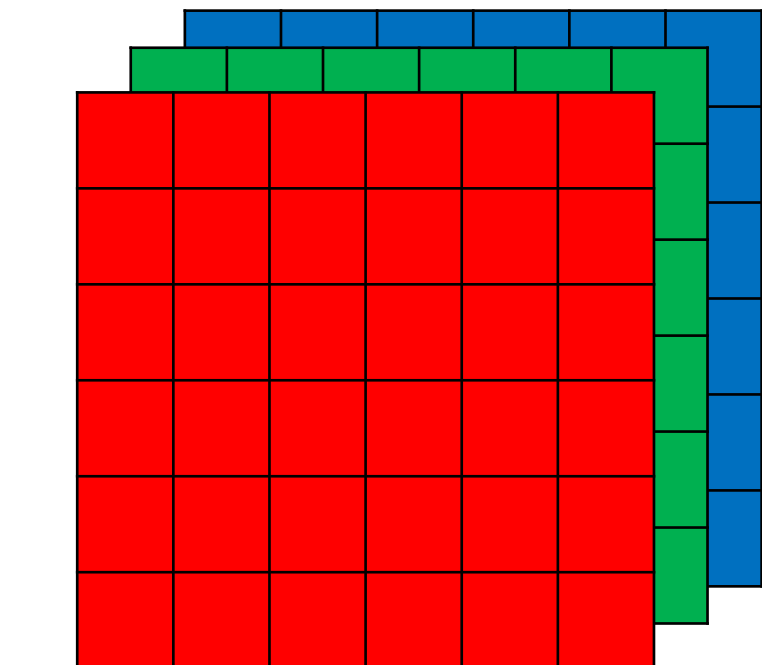


=



Summary: $n \times n \times n_c$ * $f \times f \times n_c \rightarrow n-f+1 \times n-f+1 \times n'_c$
 $6 \times 6 \times 3$ $3 \times 3 \times 3$ $4 \times 4 \times 2$ #filters

Multiple filters

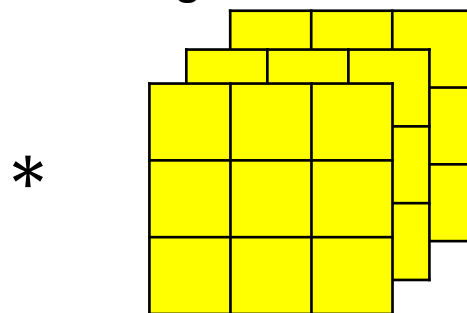


$a^{[0]}$
(input) $6 \times 6 \times 3$

$$z^{[1]} = W^{[1]}a^{[0]} + b^{[1]}$$

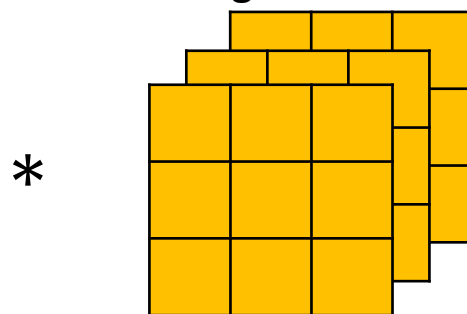
$$a^{[1]} = g(z^{[1]})$$

Vertical Edge
2 filters



$3 \times 3 \times 3$

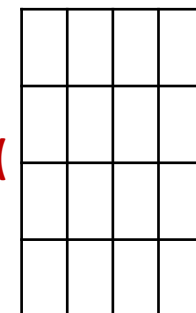
Horizontal Edge



$3 \times 3 \times 3$

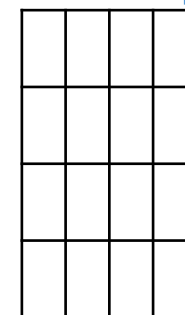
$W^{[1]}$

$W^{[1]}a^{[0]}$



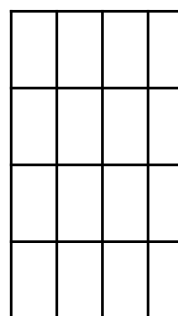
4×4

$= \text{Relu}(\quad + b_1) =$

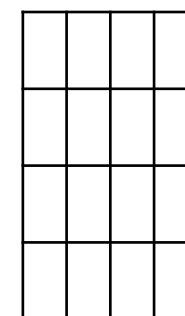


4×4

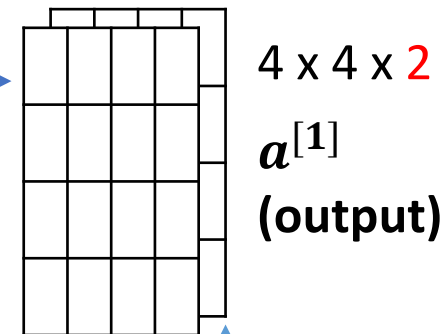
$= \text{Relu}(\quad + b_2) =$



4×4



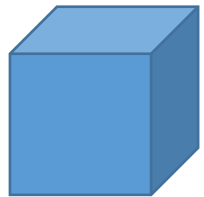
4×4



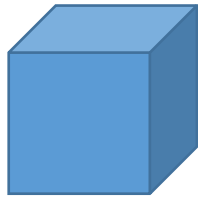
$4 \times 4 \times 2$
 $a^{[1]}$
(output)

Multiple filters

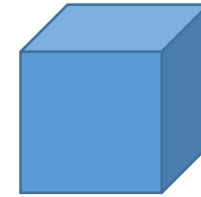
If you have 10 filters that are 3x3x3 in one layer of a neural network, how many parameters does that layer have?



1st filter



2nd filter



10th filter

3 x 3 x 3

27 parameters

+ 1 bias

-> 28 parameters for each filter

Total no of Parameters: 280

Summary of notation

If layer l (e.g., layer 1) is a convolution layer:

$f^{[l]}$ = filter size

$p^{[l]}$ = padding

$s^{[l]}$ = stride

$n_f^{[l]}$ = number of filters

Each filter is: $f^{[l]} \times f^{[l]} \times n_c^{[l-1]}$

Activations: $a^{[l]} \rightarrow n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$

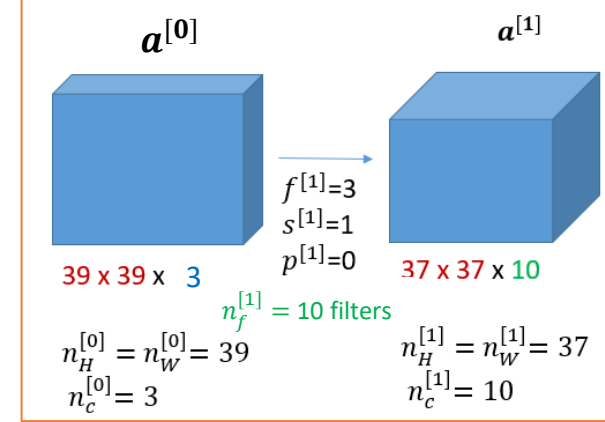
Weights: $f^{[l]} \times f^{[l]} \times n_c^{[l-1]} \times n_f^{[l]}$

Bias: $n_f^{[l]}$

#filters in layer l

Input: $n_H^{[l-1]} \times n_W^{[l-1]} \times n_c^{[l-1]}$

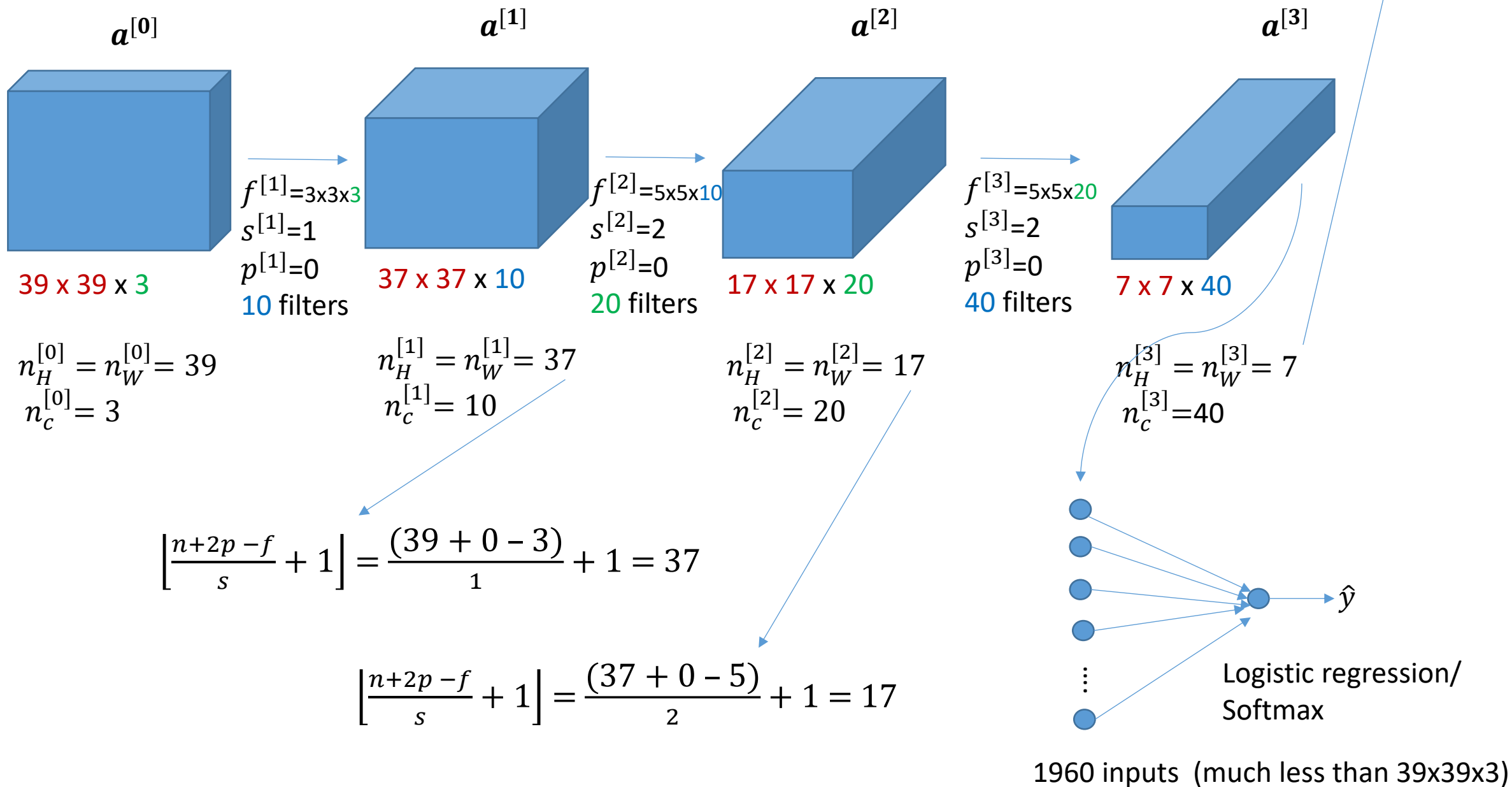
Output: $n_H^{[l]} \times n_W^{[l]} \times n_f^{[l]}$



$$= \frac{(39 + 0 - 3)}{1} + 1 = 37$$

$$n_H^{[l]} = \left\lfloor \frac{n_H^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1 \right\rfloor$$

Example ConvNet



Types of layer in a convolutional network:

- Convolution (CONV)
- Pooling (POOL)
- Fully connected (FC)

Pooling layer: Max pooling

1	3	2	1
2	9	1	1
1	3	2	3
5	6	1	2

4 x 4

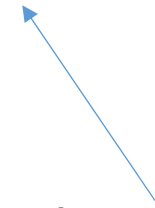


Hyper-parameters:

- $F = 2$
- $S = 2$

9	2
6	3

2 x 2



$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor = \frac{(4+0-2)}{2} + 1 = 2$$

Pooling layer: Max pooling

1	3	2	1	3
2	9	1	1	5
1	3	2	3	2
8	3	5	1	0
5	6	1	2	9

5 x 5 x **Nc**

Number of Channels

Hyper-parameters:

- F = 3
- S = 1

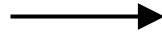
9	9	5
9	9	5
8	6	9

3 x 3 x **Nc**

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor = \frac{(5+0-3)}{1} + 1 = 3$$

Pooling layer: Average pooling

1	3	2	1
2	9	1	1
1	4	2	3
5	6	1	2



Hyper-parameters:

- $F = 2$
- $S = 2$

3.75	1.25
4	2

Summary of pooling

Hyperparameters:

f : filter size

Common values: ($f=2, s=2$) or ($f=3, s=2$)

s : stride

Max or Average pooling

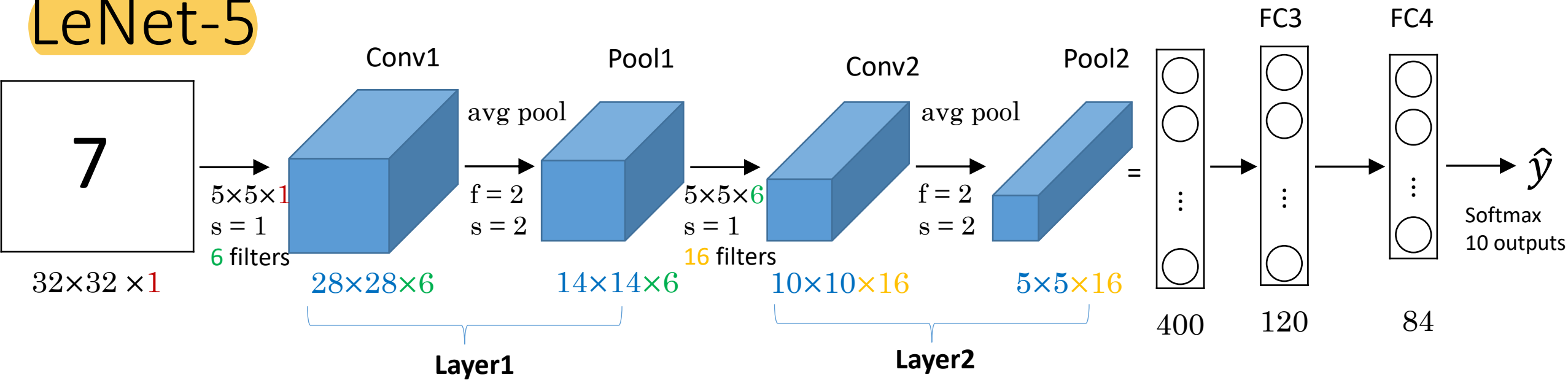
- Max pooling is commonly used.

No parameters to learn!

$$n_H \times n_W \times n_c \Rightarrow \left\lfloor \frac{n_H + 2p - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n_W + 2p - f}{s} + 1 \right\rfloor \times n_c$$

Convolutional Neural network example

LeNet-5

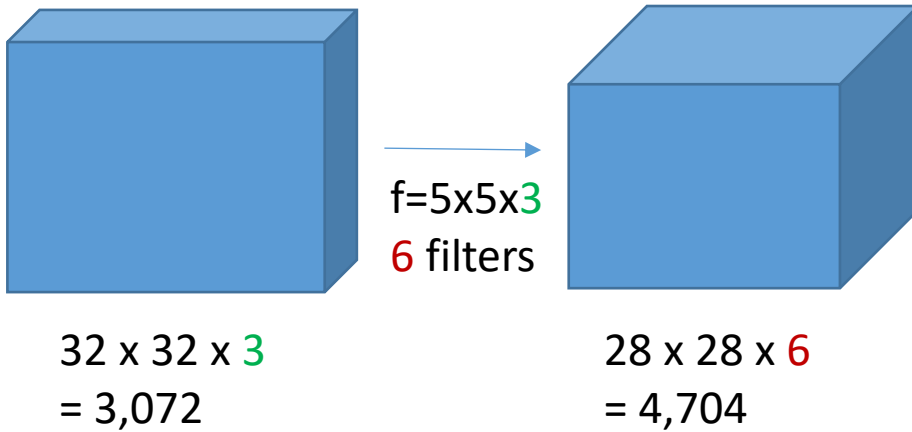


- 60k parameters
- From left to right, N_H , N_W decreases and N_C increases.
- Layers: Conv Pool Conv Pool Fc Fc output
- Uses sigmoid / tanh

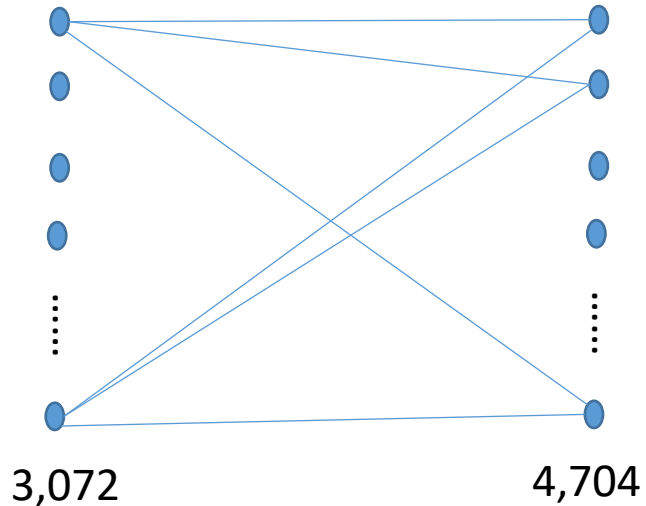
Neural network example

	Activation shape	Activation Size	# parameters
Input:	(32,32,3)	3,072	0
CONV1(f=5,s=1)	(28,28,8)	6,272	608 $((5 \times 5 \times 3 + 1) \times 8)$
POOL1 (f=2,s=2)	(14,14,8)	1,568	0
CONV2 (f=5, s=1)	(10,10,16)	1,600	3216 $((5 \times 5 \times 8 + 1) \times 16)$
POOL2 (f=2,s=2)	(5,5,16)	400	0
FC3	(120,1)	120	48120 $(120 \times 400 + 120)$
FC4	(84,1)	84	10164 $(84 \times 120 + 84)$
Softmax	(10,1)	10	850 $(10 \times 84 + 10)$

Why convolutions



$$(5 \times 5 \times 3 + 1) \times 6 = \mathbf{456} \text{ parameters}$$



$$4,704 \times 3,072 = \text{around } \mathbf{14M} \text{ parameters}$$

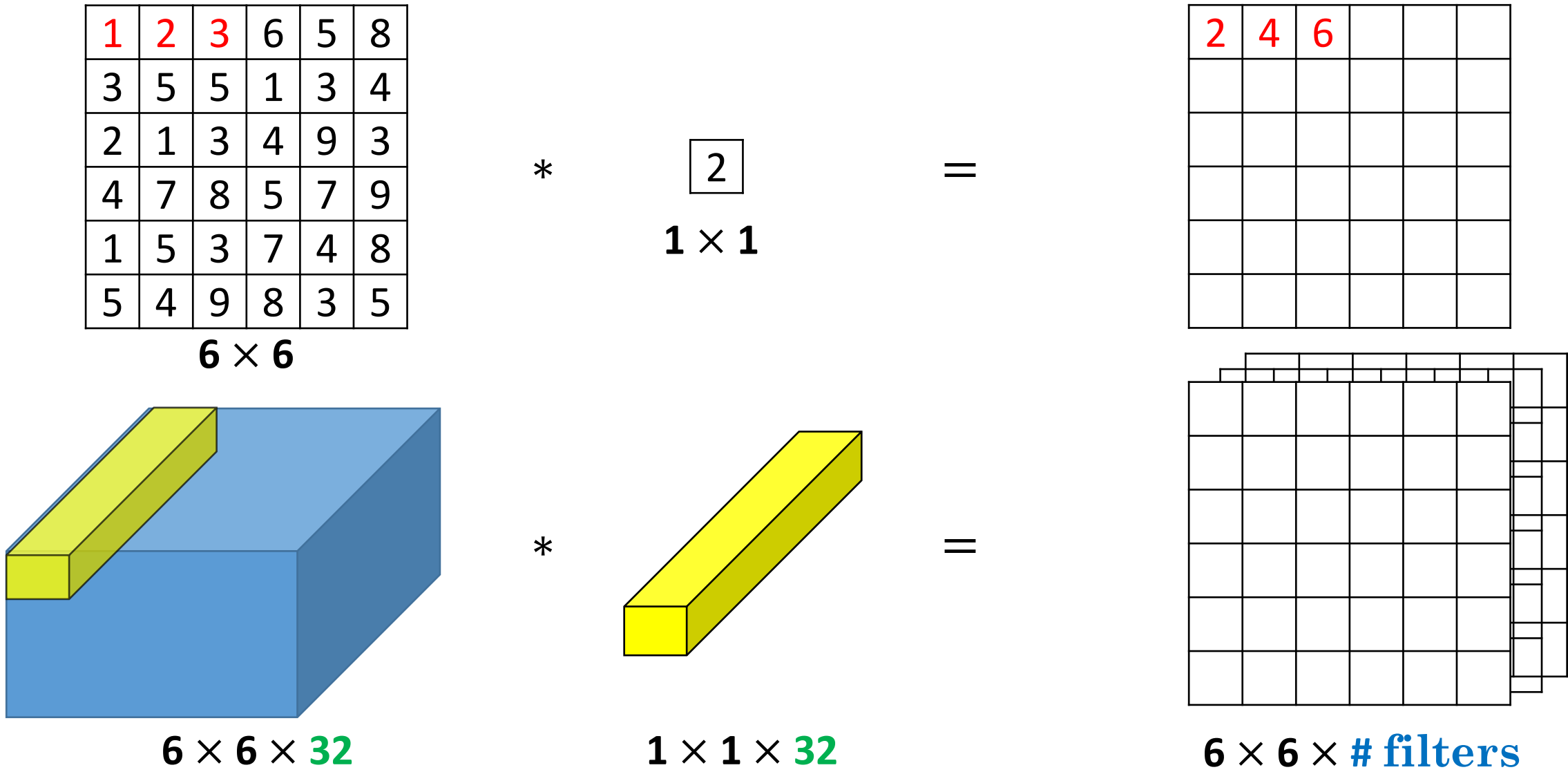
1 x 1 Convolutions

[Lin, Chen, and Yan. 2013. Network in network. arXiv:1312.4400.]

- **Does this concept make sense?!? Yes.**
- 1 x 1 convolutions, a.k.a. Network-in-network (NiN) connections, are convolutional kernels with `kernel_size=1`.
- A 1 x 1 convolution gives you a fully connected linear layer across channels!
- It can be used to map from many channels to fewer channels.
- 1 x 1 convolutions add additional neural network layers with very few additional parameters.
 - Unlike Fully Connected (FC) layers which add a lot of parameters

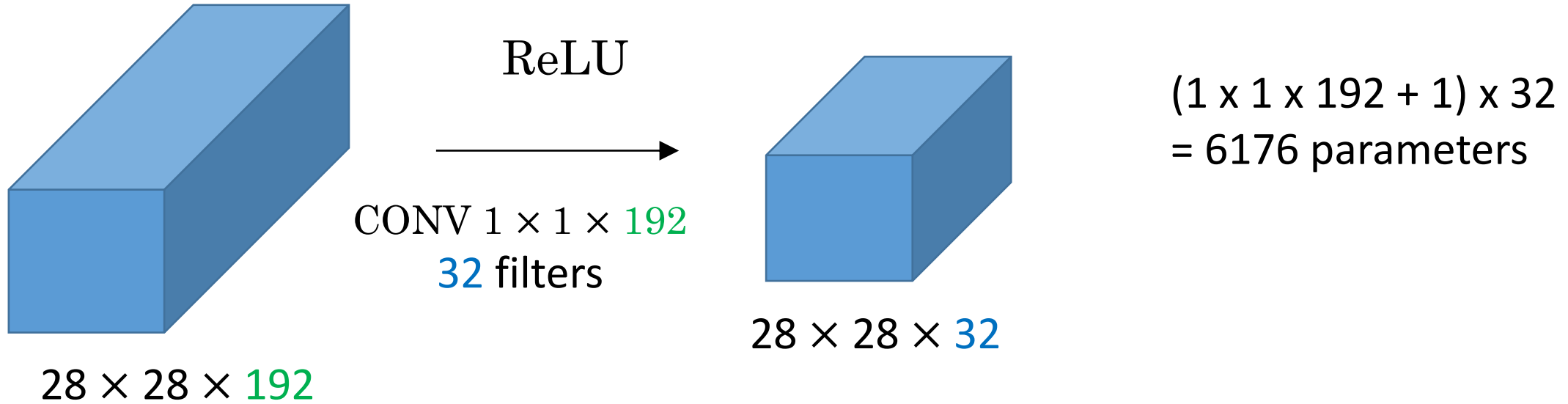
Network in Network and 1×1 convolutions

Why does a 1 × 1 convolution do?



[Lin et al., 2013. Network in network]

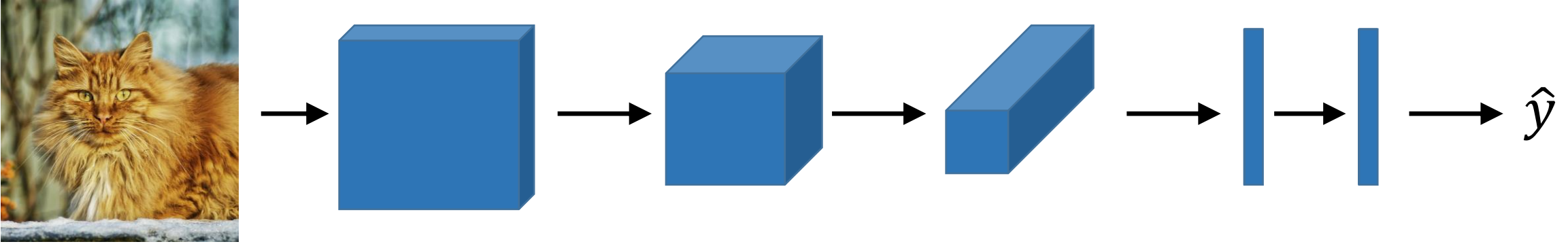
Using 1×1 convolutions



- It can be used to map from many channels to fewer channels.
- If 192 filters are used, output is $28 \times 28 \times 32$.

Putting it together

Training set $(x^{(1)}, y^{(1)}) \dots (x^{(m)}, y^{(m)})$.



$$\text{Cost } J = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

Use gradient descent to optimize parameters to reduce J .

- e.g., learn the parameters of CNN filters

Convolutional Neural Networks for text classification

A 1D convolution for text : “tentative deal reached to keep government open”

tentative	0.2	0.1	-0.3	0.4
deal	0.5	0.2	-0.3	-0.1
reached	-0.1	-0.3	-0.2	0.4
to	0.3	-0.3	0.1	0.1
keep	0.2	-0.3	0.4	0.2
government	0.1	0.2	-0.1	-0.1
open	-0.4	-0.4	0.2	0.3

t,d,r	-1.0	← tri-gram representation
d,r,t	-0.5	
r,t,k	-3.6	
t,k,g	-0.2	
k,g,o	0.3	

Apply a **filter** (or **kernel**) of size 3 (for tri-grams) 1D output

3	1	2	-3
-1	2	1	-3
1	1	-1	1

3 × 4

1D convolution for text with padding

We can tell that **tentative** is the first word in a sentence.

∅	0.0	0.0	0.0	0.0
tentative	0.2	0.1	-0.3	0.4
deal	0.5	0.2	-0.3	-0.1
reached	-0.1	-0.3	-0.2	0.4
to	0.3	-0.3	0.1	0.1
keep	0.2	-0.3	0.4	0.2
government	0.1	0.2	-0.1	-0.1
open	-0.4	-0.4	0.2	0.3
∅	0.0	0.0	0.0	0.0

∅,t,d	-0.6
t,d,r	-1.0
d,r,t	-0.5
r,t,k	-3.6
t,k,g	-0.2
k,g,o	0.3
g,o,∅	-0.5

1D output

Apply a **filter** (or **kernel**) of size 3

3	1	2	-3
-1	2	1	-3
1	1	-1	1

3 × 4

3 channel 1D convolution with padding = 1

∅	0.0	0.0	0.0	0.0
tentative	0.2	0.1	-0.3	0.4
deal	0.5	0.2	-0.3	-0.1
reached	-0.1	-0.3	-0.2	0.4
to	0.3	-0.3	0.1	0.1
keep	0.2	-0.3	0.4	0.2
government	0.1	0.2	-0.1	-0.1
open	-0.4	-0.4	0.2	0.3
∅	0.0	0.0	0.0	0.0

∅,t,d	-0.6	0.2	1.4
t,d,r	-1.0	1.6	-1.0
d,r,t	-0.5	-0.1	0.8
r,t,k	-3.6	0.3	0.3
t,k,g	-0.2	0.1	1.2
k,g,o	0.3	0.6	0.9
g,o,∅	-0.5	-0.9	0.1

Apply 3 **filters** of size 3

3	1	2	-3
-1	2	1	-3
1	1	-1	1

3 × 4

1	0	0	1
1	0	-1	-1
0	1	0	1

3 × 4

1	-1	2	-1
1	0	-1	3
0	2	2	1

3 × 4

Could also use padding = 2.
Also called “wide convolution”.

conv1d, padded with max pooling over time

∅	0.0	0.0	0.0	0.0
tentative	0.2	0.1	-0.3	0.4
deal	0.5	0.2	-0.3	-0.1
reached	-0.1	-0.3	-0.2	0.4
to	0.3	-0.3	0.1	0.1
keep	0.2	-0.3	0.4	0.2
government	0.1	0.2	-0.1	-0.1
open	-0.4	-0.4	0.2	0.3
∅	0.0	0.0	0.0	0.0

∅,t,d	-0.6	0.2	1.4
t,d,r	-1.0	1.6	-1.0
d,r,t	-0.5	-0.1	0.8
r,t,k	-3.6	0.3	0.3
t,k,g	-0.2	0.1	1.2
k,g,o	0.3	0.6	0.9
g,o,∅	-0.5	-0.9	0.1
max p	0.3	1.6	1.4

“Keep government open” is an important tri-gram in the first filter.

Apply 3 filters of size 3

3	1	2	-3
-1	2	1	-3
1	1	-1	1

1	0	0	1
1	0	-1	-1
0	1	0	1

1	-1	2	-1
1	0	-1	3
0	2	2	1

conv1d, padded **with ave pooling over time**

∅	0.0	0.0	0.0	0.0
tentative	0.2	0.1	-0.3	0.4
deal	0.5	0.2	-0.3	-0.1
reached	-0.1	-0.3	-0.2	0.4
to	0.3	-0.3	0.1	0.1
keep	0.2	-0.3	0.4	0.2
government	0.1	0.2	-0.1	-0.1
open	-0.4	-0.4	0.2	0.3
∅	0.0	0.0	0.0	0.0

∅,t,d	-0.6	0.2	1.4
t,d,r	-1.0	1.6	-1.0
d,r,t	-0.5	-0.1	0.8
r,t,k	-3.6	0.3	0.3
t,k,g	-0.2	0.1	1.2
k,g,o	0.3	0.6	0.9
g,o,∅	-0.5	-0.9	0.1
ave p	-0.87	0.26	0.53

Apply 3 **filters** of size 3

3	1	2	-3
-1	2	1	-3
1	1	-1	1

1	0	0	1
1	0	-1	-1
0	1	0	1

1	-1	2	-1
1	0	-1	3
0	2	2	1

Other less useful notions: stride = 2

∅	0.0	0.0	0.0	0.0
tentative	0.2	0.1	-0.3	0.4
deal	0.5	0.2	-0.3	-0.1
reached	-0.1	-0.3	-0.2	0.4
to	0.3	-0.3	0.1	0.1
keep	0.2	-0.3	0.4	0.2
government	0.1	0.2	-0.1	-0.1
open	-0.4	-0.4	0.2	0.3
∅	0.0	0.0	0.0	0.0

∅,t,d	-0.6	0.2	1.4
d,r,t	-0.5	-0.1	0.8
t,k,g	-0.2	0.1	1.2
g,o,∅	-0.5	-0.9	0.1

Apply 3 **filters** of size 3

3	1	2	-3
-1	2	1	-3
1	1	-1	1

1	0	0	1
1	0	-1	-1
0	1	0	1

1	-1	2	-1
1	0	-1	3
0	2	2	1

conv1d, k -max pooling over time, $k = 2$

$\emptyset$	0.0	0.0	0.0	0.0
tentative	0.2	0.1	-0.3	0.4
deal	0.5	0.2	-0.3	-0.1
reached	-0.1	-0.3	-0.2	0.4
to	0.3	-0.3	0.1	0.1
keep	0.2	-0.3	0.4	0.2
government	0.1	0.2	-0.1	-0.1
open	-0.4	-0.4	0.2	0.3
$\emptyset$	0.0	0.0	0.0	0.0

Apply 3 **filters** of size 3

3	1	2	-3
-1	2	1	-3
1	1	-1	1

1	0	0	1
1	0	-1	-1
0	1	0	1

1	-1	2	-1
1	0	-1	3
0	2	2	1

$\emptyset, t, d$	-0.6	0.2	1.4
t, d, r	-1.0	1.6	-1.0
d, r, t	-0.5	-0.1	0.8
r, t, k	-3.6	0.3	0.3
t, k, g	-0.2	0.1	1.2
k, g, o	0.3	0.6	0.9
$g, o, \emptyset$	-0.5	-0.9	0.1

Take two max values.

2-max p	-0.2	1.6	1.4
	0.3	0.6	1.2

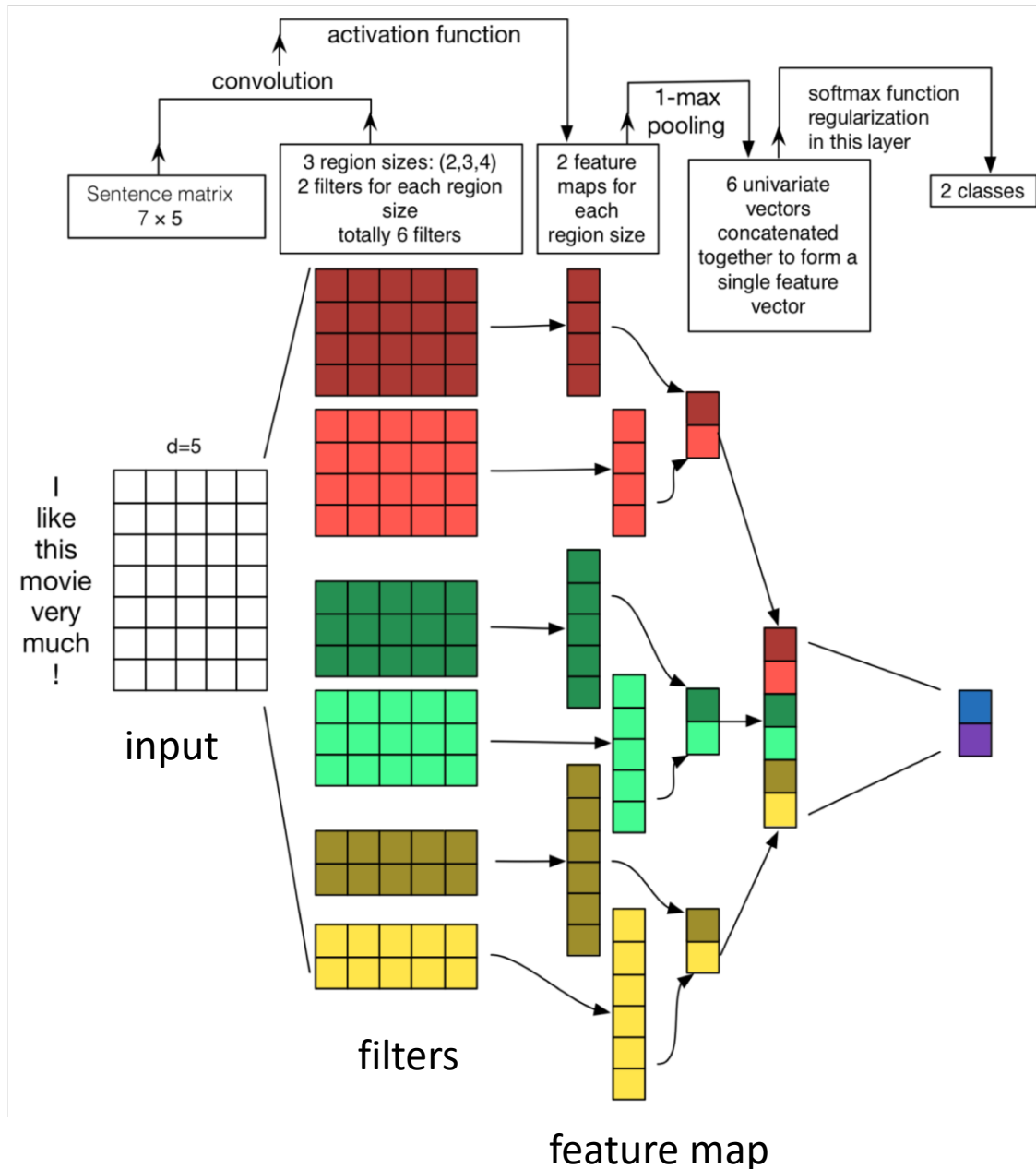
Single Layer CNN for Sentence Classification

- Yoon Kim (2014): Convolutional Neural Networks for Sentence Classification. EMNLP 2014. <https://arxiv.org/pdf/1408.5882.pdf>
Code: <https://arxiv.org/pdf/1408.5882.pdf> [Theano!, etc.]
- A variant of convolutional NNs of Collobert, Weston et al. (2011)
- Goal: Sentence classification:
 - Mainly positive or negative sentiment of a sentence
 - Other tasks like:
 - Subjective or objective language sentence
 - Question classification: about person, location, number, ...

From:
Zhang and Wallace
(2015) A Sensitivity
Analysis of (and
Practitioners' Guide
to) Convolutional
Neural Networks for
Sentence
Classification

<https://arxiv.org/pdf/1510.03820.pdf>

(follow on paper, not
famous, but a nice picture)



In PyTorch

batch_size = 16

word_embed_size = 4

seq_len = 7

input = torch.randn(batch_size, word_embed_size, seq_len) # input: (16, 4, 7)

conv1 = Conv1d(in_channels=word_embed_size, out_channels=3,
kernel_size=3) # can add padding=1

hidden1 = conv1(input) # hidden1: (16, 3, 5)

hidden2 = torch.max(hidden1, dim=2).values # max pool; hidden2: (16, 3)

	t,d,r	d,r,t	r,t,k	t,k,g	k,g,o
out_channel 1	-1.0	-0.5	-3.6	-0.2	0.3
out_channel 2	1.6	-0.1	0.3	0.1	0.6
out_channel 3	-1.0	0.8	0.3	1.2	0.9

tentative	deal	reached	to	keep	government	open
0.2	0.5	-0.1	0.3	0.2	0.1	-0.4
0.1	0.2	-0.3	-0.3	-0.3	0.2	-0.4
-0.3	-0.3	-0.2	0.1	0.4	-0.1	0.2
0.4	-0.1	0.4	0.1	0.2	-0.1	0.3

Tri-gram filter: 4x3

of filters

max p

0.3

1.6

1.2

Model comparison: Our growing toolkit

- **Bag of Vectors (Multi-Layer Neural Network):** Surprisingly good baseline for simple classification problems. Especially if followed by a few ReLU layers!
- **CNNs:** good for classification, need zero padding for shorter phrases, hard to interpret, **easy to parallelize on GPUs.**
Efficient and versatile
- **Recurrent Neural Networks:** Cognitively plausible (reading from left to right), not best for classification (if just use last state), much slower than CNNs, good for sequence tagging and classification, great for language models, can be amazing with attention mechanisms.

Reference

- Neural Network and Deep Learning, Andrew Ng,
<https://www.coursera.org/learn/neural-networks-deep-learning>
- Natural Language Processing with Deep Learning, Christopher Manning, Stanford